

# 1 Problem definition

Implement an application that manages a set of domain entities (for example: a library system, music store catalog, flower shop, F1 racing management system etc.).

The application must support multiple user roles and allow users to interact with the system according to their permissions.

You are free to choose any domain you find interesting. **The chosen domain will remain the same throughout the entire semester and will be extended in later assignments and in the final project.**

The purpose of this assignment is to implement a simple but well-structured application using object-oriented principles and a layered architecture.

**NOTE:** You may use any programming language and any technological stack (recommended: Java, C#, Python or C++ for the application and MySQL, SQL Server or PostgreSQL for persistence). Frameworks such as Spring, Hibernate, Django, ASP.NET, Node.js, or others are allowed. However, you are encouraged to implement your own data access logic (for example repository methods) instead of relying entirely on automatic framework generation.

## 1.1 Users and roles

The application must support at least three types of users:

- **Visitor** – can view public information but cannot modify data.
- **Administrator** – has full access to manage the system and perform CRUD operations on all entities (after authentication).
- **Secondary role** – a domain-specific role (requires authentication). For example:
  - seller (in a shop system)
  - librarian (in a library system)
  - employee (in a management system)

Each role should have clearly defined permissions in the use cases you define.

## 1.2 Architecture

The application must follow a **MVC (Model–View–Controller) architecture**.

The system must be organized into the following components:

- **Model**

The model represents the domain of the application and includes:

- domain entities
- repositories or data access components

- business logic implemented in services

- **View**

The view is responsible for presenting information to the user. Depending on the technology stack used, the view may be:

- web pages (HTML templates)
- a graphical user interface

This implies the need for an UI which validates the functionality of your application. Make sure you have a starting page, a login page and other pages that allow listing, filtering, editing and deleting entities. Recommended tools for Java GUI building are Java Swing and JavaFX, but any technology is allowed as long as it provides the required views.

- **Controller**

Controllers handle incoming user requests, coordinate the application flow and delegate business logic to the service layer.

Controllers must **not contain business logic or persistence logic**.

The responsibilities must be clearly separated:

- Controllers handle requests and responses
- Services implement business logic
- Repositories handle persistence
- Entities represent the domain model

**Direct database access from controllers or views is not allowed!**

### 1.3 Functional requirements

The system must support functionality for managing **ONE** core domain entity.

- **CRUD operations (1 point)**

The system must allow users with appropriate permissions to:

- Create new entities
- Read/list existing entities
- Update entities
- Delete entities

- **Role-based access control (1 point)**

The system must enforce permissions for the following roles:

- Visitors – read-only access to public information

- Administrators – full CRUD access to all entities
- Secondary role – domain-specific permissions
- **MVC architecture (2 points)**  
The project must clearly separate:
  - Controllers
  - Views
  - Models (entities, services, repositories)

Controllers must only coordinate requests and responses, while business logic must be implemented in the service layer.
- **Repository implementation (1 point)**  
Persistence must be implemented through repository components responsible for data access.  
You are encouraged to implement custom repository methods where appropriate.
- **Additional features of your choosing (2 points)**  
The system must implement **TWO** additional functionalities such as:
  - searching or filtering entities by a column (name, date added, price etc.)
  - sorting results (by name, by date added etc., both ascending and descending)
  - pagination (in UI lists)

## 2 Deliverables

- Branch/repository with the project (7 points)
- Documentation (you can either fill in the documentation template provided on Teams, or create a new document with the same structure) (3 points)

**Documentation requirements:** For each assignment/project, ensure your documentation includes **at least** the following diagrams:

- One use case diagram representing the system actors and main use cases.
- One entity-relationship diagram describing the database structure.
- One class diagram for the core domain model.
- One sequence diagram for the most important workflow in the system.

Upload the project and the documentation onto a new branch on your repository on Github. If you are using multiple repositories for this lab, you will want to merge into "main/master", otherwise you should keep the main branch empty and instead have a branch for the final version to merge the new branch into. Open a Pull Request and assign @volcovs as a reviewer.

### 3 BONUS requirements

- **Password hashing (+0.1 points)**

User passwords must be stored using a secure hashing algorithm such as BCrypt or Argon2.

- **Spring Security integration or equivalents (+0.5 points)**

If you are using the Spring ecosystem or similar you may integrate a dependency such as Spring Security to handle authentication and authorization.

### 4 DEADLINE

The deadline for this assignment is 23.03.2026! Each week of delay after the deadline will result in a penalty of one point deducted from the final grade.